# API Connector - Documentation Développeur

Sylvain Boutet

Version 1.0, 2025-01-03

# Table of Contents


Vue d'ensemble . . . 1
- Cas d'usage principaux . . . 1

Architecture du Module . . . 2
- Modèles principaux . . . 2
- Hiérarchie et héritage . . . 2

Configuration des Collections . . . 3
- Champs principaux . . . 3
- Variables de collection . . . 3
- Authentification au niveau collection . . . 3

Configuration des Connecteurs . . . 4
- Champs de base . . . 4
- Variables et paramètres . . . 4
- Corps de requête . . . 4
- Upload de fichiers . . . 4

Variables Dynamiques . . . 6
- Syntaxe d'utilisation . . . 6
- Ordre de priorité . . . 6
- Exemples pratiques . . . 6

Exécution des Appels API . . . 7
- Méthodes d'exécution . . . 7
- Variables disponibles dans les crons . . . 7
- Exemples d'exécution en cron . . . 7

Monitoring et Logs . . . 9
- Champs de statistiques . . . 9
- Logs détaillés . . . 9

Upload de Fichiers . . . 10
- Configuration . . . 10
- Exemple d'usage en cron . . . 10
- Formats supportés . . . 10

Sécurité et Bonnes Pratiques . . . 11
- Gestion des credentials . . . 11
- Gestion d'erreurs . . . 11
- Performance . . . 11

Intégrations No-Code Serveur . . . 12
- Concept révolutionnaire . . . 12
- Workflow no-code serveur . . . 12
  - Étape 1 : Configuration API (100% graphique) . . . 12
  - Étape 2 : Planification (Interface Odoo) . . . 12

Étape 3 : Traitement automatique (Interface Odoo) . . . 12
Cas d'usage no-code serveur . . . 13
Synchronisation produits e-commerce . . . 13
Import commandes clients . . . 13
Notification de livraison . . . 14
Templates d'actions automatisées . . . 14
Template : Création d'enregistrements . . . 14
Template : Mise à jour d'enregistrements . . . 14
Avantages de l'approche no-code serveur . . . 15
Limitations et recommandations . . . 15
Intégration avec Odoo . . . 16
Déclencheurs automatiques . . . 16
Modèles Odoo compatibles . . . 16
FAQ et Dépannage . . . 17
Erreurs courantes . . . 17
Logs de débogage . . . 17
Support et communauté . . . 17
Export/Import de Configurations . . . 18
Vue d'ensemble . . . 18
Export de configurations . . . 18
Utilisation de l'export . . . 18
Structure du fichier JSON exporté . . . 18
Import de configurations . . . 19
Assistant d'import . . . 19
Processus d'import . . . 19
Fonctionnalités avancées . . . 20
Cas d'usage pratiques . . . 20
Sauvegarde et restauration . . . 20
Migration entre environnements . . . 20
Partage de configurations . . . 20
Versioning et historique . . . 21
Intégration avec les crons . . . 21
Import programmatique . . . 21
Sécurité et bonnes pratiques . . . 21
Annexes . . . 23
Exemple complet : Synchronisation commandes . . . 23
Annexe : Routes d'Exemple Préconfigurées . . . 25
Collection : REST APIs Examples . . . 25
Get Posts . . . 25
Get Single Post . . . 25
Random Cat Fact . . . 25

🌐 Collection : International Public APIs . . . 25
Random User Generator . . . 25
↑ Collection : POST Request Examples . . . 26
Create New Post . . . 26
Create User Comment . . . 26
⇄ Collection : JSON-RPC Examples . . . 26
JSON-RPC Demo Call . . . 26
🖧 Collection : GraphQL Examples . . . 27
Rick and Morty GraphQL . . . 27
</> Collection : XML/SOAP Examples . . . 27
XML Demo Post . . . 27
🔒 Collection : Authentication Examples (Désactivées) . . . 27
Basic Auth Demo . . . 27
Bearer Token Demo . . . 28
🛡 Sécurité et Confidentialité . . . 28
▶ Utilisation des Exemples . . . 28
🔧 Personnalisation . . . 28

# Vue d’ensemble

Le module **API Connector** est un outil Postman-like intégré à Odoo qui permet de configurer et exécuter des appels API sortants de manière graphique et automatisée.

## Cas d’usage principaux

- **Synchronisation données** : Envoyer des commandes, factures, contacts vers des systèmes externes
- **Intégrations e-commerce** : Synchroniser stocks, prix, commandes avec des plateformes
- **Notifications automatiques** : Alerter des systèmes tiers lors d’événements Odoo
- **Upload de fichiers** : Envoyer des documents (factures PDF, bons de livraison) vers des APIs
- **Webhooks sortants** : Déclencher des actions sur des services externes

# 🏗 Architecture du Module

## Modèles principaux

| Modèle | Nom technique | Description |
|---|---|---|
| **Collections** | `api.collection` | Regroupement d'endpoints avec configuration partagée (authentification, URL de base, variables) |
| **Connecteurs** | `api.connector` | Configuration individuelle d'un endpoint API (URL, méthode, corps de requête, authentification) |
| **Logs d'appels** | `api.call.log` | Historique détaillé de tous les appels API avec temps de réponse, statut, erreurs |

## Hiérarchie et héritage

```
Collection (api.collection)
 ├─── Configuration partagée
 │    ├─── Base URL
 │    ├─── Authentification
 │    ├─── Headers par défaut
 │    └─── Variables de collection
 └─── Connecteurs (api.connector)
     ├─── Hérite de la collection (optionnel)
     ├─── Configuration spécifique
     └─── Variables d'endpoint
```

# ⚙ Configuration des Collections

## Champs principaux

| Champ | Type | Description |
|---|---|---|
| `name` | Char | Nom de la collection (ex: "API Shopify", "Webhooks Stripe") |
| `base_url` | Char | URL de base partagée (ex: "https://api.shopify.com/v1") |
| `auth_type` | Selection | Type d'authentification : None, Basic, Bearer, API Key, OAuth 2.0 |
| `collection_variables` | Text (JSON) | Variables partagées utilisables dans tous les endpoints de la collection |
| `default_headers` | Text (JSON) | Headers HTTP par défaut appliqués à tous les endpoints |

## Variables de collection

Les variables de collection permettent de centraliser des valeurs réutilisables :

```
{
    "api_version": "v2",
    "environment": "production",
    "store_id": "12345",
    "timeout": "30"
}
```

Ces variables peuvent être utilisées dans les URLs et corps de requête avec la syntaxe `{variable_name}`

## Authentification au niveau collection

| Type | Configuration requise |
|---|---|
| **None** | Aucune authentification |
| **Basic Auth** | Username + Password |
| **Bearer Token** | Token d'accès |
| **API Key** | Nom du header + Clé API |
| **OAuth 2.0** | Client ID + Client Secret + URLs d'autorisation et token |

# 🔌 Configuration des Connecteurs

## Champs de base

| Champ | Type | Description |
|---|---|---|
| `name` | Char | Nom du connecteur (ex: "Create Order", "Update Stock") |
| `collection_id` | Many2one | Collection parente (optionnel) |
| `api_type` | Selection | Type d'API : REST, JSON-RPC, GraphQL, SOAP/XML |
| `method` | Selection | Méthode HTTP : GET, POST, PUT, PATCH, DELETE |
| `base_url` | Char | URL de base (peut hériter de la collection) |
| `endpoint_path` | Char | Chemin de l'endpoint (ex: "/orders", "/products/{product_id}") |

## Variables et paramètres

| Champ | Type | Description |
|---|---|---|
| `url_parameters` | Text (JSON) | Paramètres d'URL pour les chemins dynamiques (ex: `{"product_id": "123"}`) |
| `endpoint_variables` | Text (JSON) | Variables spécifiques à cet endpoint (surchargent les variables de collection) |
| `custom_headers` | Text (JSON) | Headers HTTP personnalisés pour cet endpoint |

## Corps de requête

| Type | Utilisation |
|---|---|
| **JSON** | Données structurées, APIs REST modernes |
| **Form Data (URL-encoded)** | Formulaires web classiques |
| **Form Data (Multipart)** | Upload de fichiers + données |
| **XML** | APIs SOAP, services legacy |
| **Raw Text** | Contenu brut, CSV, autres formats |
| **Binary (Base64)** | Fichiers encodés en base64 |

## Upload de fichiers

| Champ | Type | Description |
|---|---|---|
| `attachment_ids` | Many2many | Fichiers Odoo à envoyer (ir.attachment) |
| `file_field_name` | Char | Nom du champ fichier dans le formulaire (défaut: "ufile") |

L’upload de fichiers nécessite le type de corps **"Form Data (Multipart)"**

# </> Variables Dynamiques

## Syntaxe d'utilisation

Les variables utilisent la syntaxe `{variable_name}` et peuvent être utilisées dans :

- URLs et chemins d'endpoints
- Corps de requête (JSON, XML, Raw)
- Headers personnalisés

## Ordre de priorité

```
1. Variables dynamiques (passées lors de l'exécution)
2. Variables d'endpoint (endpoint_variables)
3. Variables de collection (collection_variables)
```

## Exemples pratiques

*Variables de collection*

```
{
    "base_api": "v2",
    "store": "myshop",
    "format": "json"
}
```

*Variables d'endpoint*

```
{
    "resource": "orders",
    "action": "create"
}
```

*URL finale*

```
https://api.shopify.com/{base_api}/{store}/{resource}/{action}.{format}
→ https://api.shopify.com/v2/myshop/orders/create.json
```

# ▶ Exécution des Appels API

## Méthodes d'exécution

| Méthode | Description |
|---|---|
| **Interface graphique** | Bouton "Execute Call" dans le formulaire du connecteur |
| **Cron jobs** | Exécution programmée via les tâches planifiées Odoo |
| **Code Python** | Appel direct depuis du code métier Odoo |

## Variables disponibles dans les crons

```
# Variables Odoo standard disponibles
env          # Environnement Odoo
model        # Modèle vide
record       # Enregistrement déclencheur (peut être vide)
records      # Recordset multiple (peut être vide)
time         # Module time Python
datetime     # Module datetime Python
dateutil     # Module dateutil Python
timezone     # Module timezone Python
b64encode    # Fonction d'encodage base64
b64decode    # Fonction de décodage base64
_logger      # Logger pour les logs serveur
log          # Fonction de log dans ir.logging
UserError    # Exception pour messages utilisateur
Command      # Namespace pour commandes x2many
```

## Exemples d'exécution en cron

*Exécution simple*

```
connector = env['api.connector'].browse(123)
result = connector.execute_call()
```

*Avec variables dynamiques*

```
connector = env['api.connector'].browse(123)
result = connector.execute_call({
    'order_id': record.id,
    'customer_name': record.partner_id.name,
    'date': datetime.date.today().strftime('%Y-%m-%d')
})
```

*Traitement par lots*

```
collection = env['api.collection'].browse(456)
orders = env['sale.order'].search([('state', '=', 'sale')])

for order in orders:
    for connector in collection.connector_ids:
        result = connector.execute_call({
            'order_id': order.id,
            'total': order.amount_total
        })
        if not result.get('success'):
            _logger.error("Sync failed for order %s", order.name)
```

# Monitoring et Logs

## Champs de statistiques

| Champ | Type | Description |
|---|---|---|
| `total_calls` | Integer | Nombre total d'appels effectués |
| `success_calls` | Integer | Nombre d'appels réussis (status 2xx) |
| `success_rate` | Float | Taux de succès (0.0 à 1.0) |
| `last_call_date` | Datetime | Date du dernier appel |
| `last_response_status` | Integer | Code de statut HTTP de la dernière réponse |
| `last_response_time` | Float | Temps de réponse en secondes |
| `last_response_data` | Text | Données de la dernière réponse |

## Logs détaillés

Chaque appel API génère un enregistrement `api.call.log` contenant :

- Date et heure précises
- Utilisateur ayant déclenché l'appel
- Paramètres de la requête
- Taille de la requête et réponse
- Temps d'exécution
- Code de statut et message d'erreur éventuel

# Upload de Fichiers

## Configuration

Pour l'upload de fichiers, configurez :

1. **Type de corps** : "Form Data (Multipart)"
2. **Nom du champ** : "ufile" (ou selon l'API cible)
3. **Fichiers** : Sélectionner les attachments Odoo

## Exemple d'usage en cron

```
# Créer un attachment depuis un fichier
attachment = env['ir.attachment'].create({
    'name': 'facture.pdf',
    'datas': b64encode(pdf_content),
    'mimetype': 'application/pdf'
})

# Ajouter au connecteur et exécuter
connector = env['api.connector'].browse(789)
connector.write({'attachment_ids': [(4, attachment.id)]})
result = connector.execute_call({
    'invoice_id': invoice.id,
    'customer_ref': invoice.partner_id.ref
})
```

## Formats supportés

Le module gère automatiquement :

- **Fichier unique** : Champ `ufile`
- **Fichiers multiples** : Champs `ufile`, `ufile[1]`, `ufile[2]`, etc.
- **MIME types** : Détection automatique depuis l'attachment
- **Noms de fichiers** : Préservation des noms originaux

# Sécurité et Bonnes Pratiques

## Gestion des credentials

- Stockez les tokens et clés API de manière sécurisée
- Utilisez des variables d'environnement pour les valeurs sensibles
- Limitez les droits d'accès aux collections critiques
- Activez la vérification SSL par défaut

## Gestion d'erreurs

*Pattern recommandé*

```
try:
    connector = env['api.connector'].browse(connector_id)
    result = connector.execute_call(variables)

    if result.get('success'):
        _logger.info("API sync successful")
        # Traiter la réponse
    else:
        _logger.error("API error: %s", result.get('error'))
        # Gérer l'erreur

except Exception as e:
    _logger.error("Sync failed: %s", str(e))
    raise UserError("Synchronization failed: %s" % str(e))
```

## Performance

- Configurez des timeouts appropriés (30s par défaut)
- Utilisez les collections pour réduire la duplication
- Surveillez les logs pour identifier les appels lents
- Implémentez des retry patterns pour les APIs instables

# 🪄 Intégrations No-Code Serveur

## Concept révolutionnaire

Le module API Connector permet de créer des **intégrations complètes sans écrire de fichiers Python**. Toute la configuration se fait via l'interface Odoo standard en combinant :

- **API Connectors** : Configuration graphique des appels API
- **Crons Odoo** : Planification via l'interface standard
- **Actions automatisées** : Traitement des réponses via l'interface

## Workflow no-code serveur

*Processus en 3 étapes*

```
1. 🔌 API Connector → Appel API externe programmé
2. ⏰ Cron Odoo → Exécution périodique automatique
3. 🎯 Action automatisée → Traitement de la réponse
```

### Étape 1 : Configuration API (100% graphique)

- **URL et authentification** : Interface API Connector
- **Variables dynamiques** : Substitution automatique
- **Corps de requête** : JSON avec variables `{variable_name}`
- **Monitoring** : Logs automatiques intégrés

### Étape 2 : Planification (Interface Odoo)

*Code minimal dans le cron (via interface)*

```
# Configuration dans Paramètres > Technique > Automatisation > Actions planifiées
connector = env['api.connector'].browse(123)  # ID depuis l'interface
result = connector.execute_call({
    'date': datetime.date.today().strftime('%Y-%m-%d'),
    'company_id': env.company.id
})
```

### Étape 3 : Traitement automatique (Interface Odoo)

*Action automatisée sur api.connector (via interface)*

```
# Configuration dans Paramètres > Technique > Automatisation > Actions automatisées
# Déclencheur : "Lors de la mise à jour d'un enregistrement"
# Condition : last_response_success == True

import json

if record.last_response_data:
    response = json.loads(record.last_response_data)

    # Traiter la réponse selon le type d'intégration
    if record.name == 'Sync Products':
        for product_data in response.get('products', []):
            env['product.product'].create({
                'name': product_data['name'],
                'default_code': product_data['sku'],
                'list_price': product_data['price']
            })

    elif record.name == 'Import Orders':
        for order_data in response.get('orders', []):
            partner = env['res.partner'].search([('email', '=', order_data
['customer_email'])], limit=1)
            if partner:
                env['sale.order'].create({
                    'partner_id': partner.id,
                    'amount_total': order_data['total']
                })
```

# Cas d'usage no-code serveur

## Synchronisation produits e-commerce

| Composant | Configuration | Résultat |
|---|---|---|
| **API Connector** | URL: `https://api.shopify.com/products` | Récupère catalogue produits |
| **Cron quotidien** | Exécution à 2h du matin | Appel API automatique |
| **Action automatisée** | Déclencheur sur `last_response_data` | Création/MAJ produits Odoo |

## Import commandes clients

| Composant | Configuration | Résultat |
|---|---|---|
| **API Connector** | URL: `https://api.woocommerce.com/orders` | Récupère nouvelles commandes |

| Composant | Configuration | Résultat |
|---|---|---|
| **Cron horaire** | Exécution toutes les heures | Vérification commandes |
| **Action automatisée** | Condition : nouvelles commandes | Création `sale.order` Odoo |

### Notification de livraison

| Composant | Configuration | Résultat |
|---|---|---|
| **API Connector** | URL: `https://api.carrier.com/notify` | Notification transporteur |
| **Action automatisée** | Déclencheur : `stock.picking` validé | Appel API automatique |
| **Réponse** | Confirmation de réception | MAJ statut livraison |

# Templates d'actions automatisées

## Template : Création d'enregistrements

*Action automatisée générique*

```
# Condition : last_response_success == True and 'create_' in record.name
import json

if record.last_response_data:
    response = json.loads(record.last_response_data)

    # Mapping générique depuis endpoint_variables
    mapping_config = json.loads(record.endpoint_variables or '{}')
    target_model = mapping_config.get('target_model', 'res.partner')
    field_mapping = mapping_config.get('field_mapping', {})

    for item in response.get('data', []):
        values = {}
        for odoo_field, api_field in field_mapping.items():
            if api_field in item:
                values[odoo_field] = item[api_field]

        if values:
            env[target_model].create(values)
            _logger.info(f"Created {target_model} from API response")
```

## Template : Mise à jour d'enregistrements

*Action automatisée de mise à jour*

```
# Condition : last_response_success == True and 'update_' in record.name
import json

if record.last_response_data:
    response = json.loads(record.last_response_data)
    mapping_config = json.loads(record.endpoint_variables or '{}')

    target_model = mapping_config.get('target_model')
    search_field = mapping_config.get('search_field', 'id')

    for item in response.get('data', []):
        search_value = item.get(search_field)
        if search_value:
            existing_record = env[target_model].search([(search_field, '=',
search_value)], limit=1)
            if existing_record:
                update_values = {}
                for odoo_field, api_field in mapping_config.get('field_mapping',
{}).items():
                    if api_field in item:
                        update_values[odoo_field] = item[api_field]

                if update_values:
                    existing_record.write(update_values)
```

# Avantages de l'approche no-code serveur

**Pour les utilisateurs métier :** * Configuration entièrement graphique * Pas de développement custom requis * Réutilisation de templates existants * Maintenance via l'interface Odoo

**Pour les développeurs :** * Création de templates réutilisables * Moins de code custom à maintenir * Standardisation des intégrations * Focus sur la logique métier complexe

# Limitations et recommandations

**Cette approche convient pour :** * Intégrations simples (CRUD basique) * Données structurées prévisibles * Transformations légères

**Utilisez du code custom pour :** * Logique métier complexe * Transformations de données avancées * Gestion d'erreurs sophistiquée * Performance critique

# Intégration avec Odoo

## Déclencheurs automatiques

Le module peut être intégré avec :

- **Cron jobs** : Exécution programmée
- **Actions automatisées** : Traitement des réponses (no-code serveur)
- **Workflows Odoo** : Actions automatiques sur les modèles
- **Webhooks entrants** : Via le module `json_rpc_api` complémentaire
- **Actions serveur** : Boutons dans les vues

## Modèles Odoo compatibles

Le module peut synchroniser données depuis/vers tous les modèles Odoo :

- `sale.order` : Commandes
- `account.move` : Factures
- `stock.picking` : Livraisons
- `res.partner` : Contacts
- `product.product` : Produits
- Et tous les modèles custom

# ⍰ FAQ et Dépannage

## Erreurs courantes

| Erreur | Cause | Solution |
|---|---|---|
| **Timeout** | API lente ou indisponible | Augmenter le timeout, vérifier l'API |
| **SSL Error** | Certificat invalide | Désactiver verify_ssl temporairement |
| **401 Unauthorized** | Authentification incorrecte | Vérifier tokens et credentials |
| **Variables non substituées** | Syntaxe incorrecte | Utiliser `{variable}` et vérifier l'orthographe |

## Logs de débogage

Pour activer les logs détaillés :

```
import logging
_logger = logging.getLogger(__name__)
_logger.setLevel(logging.DEBUG)
```

## Support et communauté

- **Documentation** : Consultez les onglets Documentation dans l'interface
- **Logs** : Menu "Call Logs" pour l'historique détaillé
- **Tests** : Utilisez le bouton "Execute Call" pour tester
- **Variables** : Vérifiez la substitution dans les logs

# 📥 Export/Import de Configurations

## Vue d'ensemble

Le module API Connector permet d'exporter et d'importer des configurations au format JSON pour faciliter la sauvegarde, le partage et la migration des configurations entre environnements.

## Export de configurations

| Type | Méthode | Contenu exporté |
|---|---|---|
| **Connecteur** | Bouton "Export Config" dans le formulaire | Configuration complète du connecteur + référence à la collection |
| **Collection** | Bouton "Export Config" dans le formulaire | Configuration de la collection + tous les connecteurs associés |

### Utilisation de l'export

*Depuis l'interface*

```
1. Ouvrir un connecteur ou une collection
2. Cliquer sur "Export Config" dans la zone des boutons
3. Le fichier JSON se télécharge automatiquement
```

### Structure du fichier JSON exporté

*Exemple d'export de connecteur*

```json
{
  "name": "Create Order API",
  "api_type": "rest",
  "method": "POST",
  "endpoint_path": "/orders",
  "auth_type": "bearer",
  "auth_token": "sk_test_xxxxx",
  "request_body_type": "json",
  "request_body": "{\"order\": {\"total\": \"{total}\"}}",
  "collection_name": "Shopify API",
  "_metadata": {
    "export_date": "2025-01-03T10:30:00",
    "export_user": "Admin",
    "odoo_version": "18.0",
    "module_version": "1.0.0",
    "type": "api_connector"
  }
}
```

*Exemple d'export de collection*

```
{
  "name": "Shopify API",
  "base_url": "https://myshop.myshopify.com/admin/api/2023-01",
  "auth_type": "bearer",
  "auth_token": "shpat_xxxxx",
  "collection_variables": "{\"format\": \"json\"}",
  "connectors": [
    {
      "name": "Get Products",
      "method": "GET",
      "endpoint_path": "/products.{format}",
      "use_collection_url": true
    },
    {
      "name": "Create Order",
      "method": "POST",
      "endpoint_path": "/orders.{format}",
      "request_body_type": "json"
    }
  ],
  "_metadata": {
    "export_date": "2025-01-03T10:30:00",
    "export_user": "Admin",
    "odoo_version": "18.0",
    "module_version": "1.0.0",
    "type": "api_collection",
    "connectors_count": 2
  }
}
```

# Import de configurations

## Assistant d'import

L'import utilise un assistant graphique accessible via : - **Menu** : Configuration → Import Configuration - **Bouton** dans les listes de connecteurs/collections

## Processus d'import

*Étapes d'import*

```
1. Sélectionner le fichier JSON exporté
2. Prévisualisation automatique du contenu
3. Choisir le type de configuration (auto-détecté)
4. Pour les connecteurs : choisir la collection cible (optionnel)
5. Cliquer "Import Configuration"
6. Redirection vers l'enregistrement créé
```

### Fonctionnalités avancées

- **Auto-détection** : Le type de configuration est détecté automatiquement
- **Validation** : Vérification de l'intégrité des données avant import
- **Recherche intelligente** : Les collections sont retrouvées par nom
- **Héritage automatique** : Les connecteurs importés héritent de leur collection
- **Gestion d'erreurs** : Messages d'erreur explicites en cas de problème

## Cas d'usage pratiques

### Sauvegarde et restauration

*Workflow de sauvegarde*

```
# 1. Export manuel depuis l'interface
# 2. Stockage des fichiers JSON
# 3. Import en cas de besoin de restauration
```

### Migration entre environnements

*Environnement de développement → Production*

```
1. Exporter les collections depuis DEV
2. Transférer les fichiers JSON
3. Importer dans l'environnement PROD
4. Ajuster les URLs et tokens si nécessaire
```

### Partage de configurations

*Distribution de configurations prêtes à l'emploi*

```
1. Créer une collection avec tous les endpoints d'une API
2. Exporter la configuration complète
3. Partager le fichier JSON avec l'équipe
4. Import en un clic dans chaque environnement
```

### Versioning et historique

*Gestion des versions*

```
1. Export avant modifications importantes
2. Nommage avec versions (ex: shopify_v1.2.json)
3. Stockage dans un système de contrôle de version
4. Rollback possible en cas de problème
```

## Intégration avec les crons

### Import programmatique

*Import via code Python*

```python
# Import d'un connecteur
import json

config_data = {
    "name": "Auto Generated Connector",
    "api_type": "rest",
    "method": "POST",
    "endpoint_path": "/webhook",
    "_metadata": {
        "type": "api_connector",
        "export_date": "2025-01-03T10:30:00",
        "export_user": "System",
        "odoo_version": "18.0",
        "module_version": "1.0.0"
    }
}

connector_id = env['api.connector'].import_config(config_data, collection_id=123)
```

*Import d'une collection complète*

```python
# Lecture depuis un fichier
with open('/path/to/collection_export.json', 'r') as f:
    config_data = json.load(f)

result = env['api.collection'].import_config(config_data)
```

## Sécurité et bonnes pratiques

**Données sensibles dans les exports :** * Les tokens et mots de passe sont inclus dans l'export * Stockez les fichiers JSON de manière sécurisée * Utilisez des variables d'environnement pour les credentials * Révisez les exports avant partage externe

**Recommandations :** * **Nommage** : Utilisez des noms explicites (api_shopify_prod_2025.json) * **Documentation** : Ajoutez des commentaires dans les descriptions * **Tests** : Testez les imports sur un environnement de test * **Sauvegarde** : Exportez régulièrement vos configurations critiques

# 🚀 Annexes

## Exemple complet : Synchronisation commandes

*Configuration collection*

```
# Collection "Shopify API"
base_url: "https://myshop.myshopify.com/admin/api/2023-01"
auth_type: "Bearer Token"
auth_token: "shpat_xxxxx"
collection_variables: {
    "format": "json",
    "fields": "id,name,total_price,customer"
}
```

*Configuration connecteur*

```
# Connecteur "Create Order"
endpoint_path: "/orders.{format}"
method: "POST"
request_body_type: "json"
endpoint_variables: {
    "source": "odoo_erp"
}
```

*Corps de requête JSON*

```
{
    "order": {
        "line_items": [
            {
                "variant_id": "{product_variant_id}",
                "quantity": "{quantity}",
                "price": "{unit_price}"
            }
        ],
        "customer": {
            "email": "{customer_email}",
            "first_name": "{customer_firstname}",
            "last_name": "{customer_lastname}"
        },
        "source_name": "{source}",
        "note": "Order created from Odoo on {creation_date}"
    }
}
```

*Exécution en cron*

```
# Cron quotidien de synchronisation
for order in env['sale.order'].search([('state', '=', 'sale'), ('synced', '=',
False)]):
    connector = env['api.connector'].browse(123)  # ID du connecteur

    result = connector.execute_call({
        'product_variant_id': order.order_line[0].product_id.shopify_variant_id,
        'quantity': order.order_line[0].product_uom_qty,
        'unit_price': order.order_line[0].price_unit,
        'customer_email': order.partner_id.email,
        'customer_firstname': order.partner_id.firstname,
        'customer_lastname': order.partner_id.lastname,
        'creation_date': order.date_order.strftime('%Y-%m-%d')
    })

    if result.get('success'):
        order.write({'synced': True, 'shopify_order_id': result.get('data', {}).get
('id')})
        _logger.info("Order %s synchronized to Shopify", order.name)
    else:
        _logger.error("Failed to sync order %s: %s", order.name, result.get('error'))
```

# Annexe : Routes d'Exemple Préconfigurées

Le module API Connector inclut un ensemble de routes d'exemple préconfigurées pour faciliter la prise en main et démontrer les différents types d'APIs supportés.

## Collection : REST APIs Examples

**Base URL** : `https://jsonplaceholder.typicode.com`

### Get Posts

| | |
|---|---|
| Type | REST GET |
| Endpoint | `/posts` |
| Description | Récupère la liste complète des articles de démonstration |
| Réponse | Array JSON avec id, title, body, userId |

### Get Single Post

| | |
|---|---|
| Type | REST GET |
| Endpoint | `/posts/:id` |
| Paramètres | `{"id": "1"}` |
| Description | Récupère un article spécifique par son ID |
| Réponse | Objet JSON avec les détails de l'article |

### Random Cat Fact

| | |
|---|---|
| Type | REST GET |
| URL | `https://catfact.ninja/fact` |
| Description | Génère un fait amusant aléatoire sur les chats |
| Réponse | `{"fact": "…", "length": 123}` |
| Usage | Démonstration d'API externe simple et fiable |

## Collection : International Public APIs

### Random User Generator

| | |
|---|---|
| Type | REST GET |
| URL | `https://randomuser.me/api/` |

| | |
|---|---|
| Description | Génère un utilisateur fictif complet |
| Réponse | Données complètes (nom, photo, adresse, contact) |
| Usage | Parfait pour les tests et démonstrations |

# ↑ Collection : POST Request Examples

**Base URL** : `https://jsonplaceholder.typicode.com`

### Create New Post

| | |
|---|---|
| Type | REST POST |
| Endpoint | `/posts` |
| Body Type | JSON |
| Corps | `{"title": "Demo Post", "body": "This is a demo post content", "userId": 1}` |
| Description | Crée un nouvel article via requête POST JSON |
| Réponse | Objet créé avec ID généré |

### Create User Comment

| | |
|---|---|
| Type | REST POST |
| Endpoint | `/comments` |
| Body Type | JSON |
| Corps | `{"name": "Demo User", "email": "demo@example.com", "body": "This is a demo comment", "postId": 1}` |
| Description | Crée un commentaire associé à un article |
| Réponse | Commentaire créé avec ID généré |

# ⇄ Collection : JSON-RPC Examples

### JSON-RPC Demo Call

| | |
|---|---|
| Type | JSON-RPC |
| URL | `https://jsonplaceholder.typicode.com/posts` |
| Body Type | JSON |
| Corps | `{"jsonrpc": "2.0", "method": "get_demo_data", "params": {"format": "json", "count": 5}, "id": 1}` |
| Description | Démontre le protocole JSON-RPC avec méthode et paramètres |
| Usage | Illustration du format JSON-RPC standard |

# Collection : GraphQL Examples

## Rick and Morty GraphQL

| | |
|---|---|
| Type | GraphQL |
| URL | `https://rickandmortyapi.com/graphql` |
| Body Type | JSON |
| Query | `{ character(id: 1) { name status species } }` |
| Description | Requête GraphQL sur l'API Rick and Morty |
| Réponse | `{"data": {"character": {"name": "Rick Sanchez", "status": "Alive", "species": "Human"}}}` |
| Usage | Démonstration des requêtes GraphQL structurées |

# Collection : XML/SOAP Examples

## XML Demo Post

| | |
|---|---|
| Type | SOAP/XML |
| URL | `https://jsonplaceholder.typicode.com/posts` |
| Body Type | XML |
| Corps | Structure XML produit avec nom, prix, catégorie |
| Description | Envoie des données XML via requête POST |
| Headers | `Content-Type: application/xml` |
| Usage | Illustration des échanges XML/SOAP |

# Collection : Authentication Examples (Désactivées)

Ces routes sont désactivées par défaut car elles nécessitent des tokens valides.

## Basic Auth Demo

| | |
|---|---|
| Type | REST GET avec Basic Auth |
| URL | `https://jsonplaceholder.typicode.com/posts/1` |
| Auth | Basic (demo_user / demo_pass) |
| Description | Démonstration de l'authentification Basic |
| Status | Désactivé (active=False) |

### Bearer Token Demo

| | |
|---|---|
| Type | REST GET avec Bearer Token |
| URL | `https://jsonplaceholder.typicode.com/posts/1` |
| Auth | Bearer (demo_bearer_token_123) |
| Description | Démonstration de l'authentification Bearer |
| Status | Désactivé (active=False) |

## Sécurité et Confidentialité

**Toutes les APIs d'exemple sont sécurisées pour les démonstrations publiques :**

- **Aucune donnée personnelle exposée** : Les APIs ne retournent pas d'informations sur l'appelant
- **Pas d'adresse IP révélée** : Contrairement à httpbin.org, ces APIs ne loggent pas les détails de connexion
- **APIs publiques légitimes** : Services de test officiels et APIs communautaires reconnues
- **Données fictives uniquement** : Tous les exemples utilisent des données de démonstration

## Utilisation des Exemples

**Pour utiliser ces exemples :**

1. **Installation** : Les routes sont créées automatiquement lors de l'installation du module
2. **Navigation** : Menu API Connector → Collections → Sélectionner une collection
3. **Test** : Cliquer sur un connecteur et utiliser le bouton "Execute Call"
4. **Personnalisation** : Dupliquer et modifier selon vos besoins
5. **Production** : Remplacer les URLs de test par vos APIs réelles

## Personnalisation

Pour adapter ces exemples à vos besoins :

```
# Modifier une route existante
connector = self.env['api.connector'].search([('name', '=', 'Random Cat Fact')])
connector.write({
    'base_url': 'https://votre-api.com/endpoint',
    'custom_headers': '{"Authorization": "Bearer YOUR_TOKEN"}',
    'active': True
})

# Créer une nouvelle route basée sur un exemple
new_connector = connector.copy({
    'name': 'Mon API Personnalisée',
    'base_url': 'https://mon-api.com/data',
    'collection_id': collection.id
})
```

---

**Version**: 1.0 - Module API Connector pour Odoo 18 **Auteur**: Sylvain Boutet - IROKOO **Date**: 2025-01-03